

AI Operations (AIOps) — Module 1 Assignment

Rohan Sai P. | Roll Number: DA24B049

Question 1 — Technical Debt Diagnosis

1. Identifying hidden technical debt category of each scenario

(a) Rounding-logic change to estimated delivery time degraded the unrelated favorite restaurants feature.

Category: Entanglement (CACE — “Changing Anything Changes Everything”). A change in the rounding logic of estimated delivery time has affected a completely unrelated favorite restaurants feature, which is a case of CACE. Every input feature of a model is optimised jointly against a shared loss, so no feature has an isolated contribution, and a change in one of the features causes the weights of the global model to change. The rounding logic change affected the weights carrying the favorite-restaurants signal and shifted them away from the values that best captured preference.

(b) A marketing dashboard team silently consumes the model’s raw output table.

Category: Undeclared consumers. An unregistered downstream dependency has formed through a shared data store. In this case other teams silently consume a model’s output via a shared table, creating invisible dependencies that block safe changes. The marketing team has built a dashboard on top of the model’s prediction table without notifying anyone, and the ML team has no way to detect the dependency since reading a table leaves no trace in the codebase. Any future change to the output’s schema or semantics now risks silently breaking a system the ML team does not know exists.

(c) The training pipeline is a chain of 14 undocumented shell scripts with no orchestration tool.

Category: Configuration and glue-code debt. This is a case of ‘pipeline jungles’. The pipeline executes but is unreconstructible: there is no explicit dependency graph and no single correct source for hyperparameters and paths. The execution order survives only as tribal knowledge, so reproducing or safely modifying the pipeline is difficult for anyone other than the person who originally wrote it.

2. Proposed mitigation for (c)

Replace the 14 undocumented shell scripts with an **MLflow Project** with a declared environment, and make the inter-stage dependency graph explicit with a **DVC pipeline**. Each script becomes a named entry point in an `MLproject` file and parameters are recorded, so the command needed to run any stage is written down rather than remembered:

```
name: food-delivery-recsys
python_env: python_env.yaml
entry_points:
  featurize:
    parameters:
      raw_path: {type: str, default: "data/raw"}
    command: "python featurize.py --raw_path {raw_path}"
  train:
    parameters:
      lr: {type: float, default: 0.001}
      batch_size: {type: int, default: 32}
    command: "python train.py --lr {lr} --batch_size {batch_size}"
```

`python_env.yaml` records the interpreter and library versions. Any team can then reproduce a stage directly from the Git URL:

```
mlflow run . -e train -P lr=0.01
```

We declare the parameters used in model configuration in one file, and the run becomes reproducible on any machine from the repository alone. Adding a `dvc.yaml` to this declares the dependencies and outputs of each stage, so DVC can then determine which stages actually need re-running when an input changes.

Question 2 — MLflow Experiment Comparison

1. Setup

I changed the starter script from the Lecture 3 exercises in two ways: the predictor was changed from `RandomForestClassifier` to `MLPClassifier`, and the dataset from IRIS to MNIST (`fetch_openml("mnist_784")`, pixel values scaled to $[0, 1]$, stratified subsample of 10,000 training and 2,000 validation images). Six runs were executed over a 3×2 grid varying learning rate (0.0001, 0.001, 0.01) and batch size (32, 128), with all other settings held fixed (one hidden layer of 128 units, 20 epochs, seed 42).

2. Written analysis

Across six runs varying learning rate and batch size over 20 epochs, the best-performing run was **mlp-lr0.001-bs32**, reaching **95.10%** validation accuracy with a final training loss of 0.0108.

Overfitting is clearly visible in that same run. Between epochs 6 and 19 the training loss fell $6.3\times$, from 0.0678 to 0.0108, while validation accuracy oscillated within a narrow band.

Learning rate had the larger effect. Both $lr = 0.0001$ runs were still improving at epoch 19, with training losses of 0.17 and 0.25, indicating underfitting; $lr = 0.01$ with batch size 32 peaked at epoch 4 and then degraded, finishing below its best at 93.75%.

3. Logging code added to the starter script

```
mlflow.log_param("lr", args.lr)
mlflow.log_param("batch_size", args.batch_size)
mlflow.log_param("hidden_size", args.hidden_size)
mlflow.log_param("epochs", args.epochs)
mlflow.log_param("n_train", args.n_train)
mlflow.log_param("seed", args.seed)

for epoch in range(args.epochs):
    model.fit(X_train, y_train)
    train_loss = model.loss_
    val_acc = accuracy_score(y_val, model.predict(X_val))
    mlflow.log_metric("train_loss", train_loss, step=epoch)
    mlflow.log_metric("val_accuracy", val_acc, step=epoch)

mlflow.log_metric("final_val_accuracy", val_acc)
```

`MLPClassifier` is coded with `max_iter=1` and `warm_start=True` so that each `fit()` call advances training by exactly one epoch from the current weights, this way logging loop can be recorded.

